

Development of an ESP32-Based Bluetooth Remote Control Vehicle: Phase One

Project Documentation

March 31, 2026

1 Introduction

The objective of this project is to design and implement a 4-wheel drive (4WD) robotic vehicle controlled wirelessly via Bluetooth. By utilizing the ESP32 microcontroller, the system benefits from integrated dual-mode Bluetooth and Wi-Fi capabilities, providing a robust platform for real-time motor control and future IoT expansions.

2 Component Specifications

The system architecture consists of the following hardware components:

- **ESP32 Microcontroller:** Serves as the central processing unit for Bluetooth communication and PWM signal generation.
- **L298N Motor Driver:** A dual H-bridge driver used to control the direction and speed of the DC motors.
- **4 DC Gear Motors and Wheels:** Provides the mechanical propulsion for the chassis.
- **2x 18650 Li-ion Batteries:** Provides a high-density power source (approx. 7.4V to 8.4V) to drive both the motors and the logic circuit.
- **Chassis:** A lightweight cardboard-based structure for component mounting.
- **Interconnects:** Jumper wires and an optional breadboard for circuit routing.

3 System Design and Internal Control

3.1 Control Logic

The vehicle is controlled via a mobile application that transmits ASCII characters over a Bluetooth Serial link. The ESP32 parses these characters to execute specific movement functions:

Direction	Input	Motor Logic (IN1, IN2, IN3, IN4)
Forward	'F'	HIGH, LOW, HIGH, LOW
Backward	'B'	LOW, HIGH, LOW, HIGH
Left	'L'	HIGH, LOW, HIGH, LOW (Differential Speed)
Right	'R'	HIGH, LOW, HIGH, LOW (Differential Speed)
Stop	'S'	LOW, LOW, LOW, LOW

Table 1: Primary Movement Control Logic

3.2 Pulse Width Modulation (PWM) Theory

To achieve variable speed, the ESP32 modulates the duty cycle of the enable pins (ENA and ENB). The average voltage delivered to the motors is calculated by:

$$V_{avg} = V_{max} \times \frac{DutyCycle}{255}$$

Values sent from the app ('1'-'9', 'q') map to a PWM range of 150 to 255. A minimum threshold of 150 is maintained to ensure the motors overcome static friction and gear resistance.

4 Hardware Implementation

4.1 Wiring and Pin Configuration

The ESP32 pins are assigned as follows:

- **PWM Control:** Pins 33 (Right) and 32 (Left).
- **Directional Control:** Pins 13 (RF), 26 (LF), 12 (RB), and 27 (LB).

4.2 Power Management

The system utilizes a common ground architecture. The 18650 batteries provide power to the L298N's power input, while the L298N's onboard regulator provides 5V back to the ESP32 (if within voltage limits) or the ESP32 is powered via a separate 3.3V regulation stage to prevent electrical noise from the motors causing logic resets.

5 Firmware Implementation

The following code demonstrates the Bluetooth serial parsing and motor execution logic.

```

1 #include <BluetoothSerial.h>
2 BluetoothSerial ESP_BT;
3
4 #define r_pwm 33
5 #define l_pwm 32
6 #define RF 13
7 #define LF 26
8 #define RB 12
9 #define LB 27

```

```

10 int incoming;
11 int speed = 150;
12 void stopC(){
13   digitalWrite(RF, LOW);
14   digitalWrite(LB, LOW);
15   digitalWrite(LF, LOW);
16   digitalWrite(RB, LOW);
17 }
18 void setup(){
19   Serial.begin(9600);
20   ESP_BT.begin("roboS");
21   pinMode(RF, OUTPUT);
22   pinMode(LF, OUTPUT);
23   pinMode(LB, OUTPUT);
24   pinMode(RB, OUTPUT);
25   digitalWrite(RF, LOW);
26   digitalWrite(LB, LOW);
27   digitalWrite(LF, LOW);
28   digitalWrite(RB, LOW);
29   Serial.begin(115200);
30 }
31 void loop(){
32   if(ESP_BT.available()){
33     incoming = ESP_BT.read();
34     if(incoming == 'S'){ // stop all
35       digitalWrite(RF,LOW);
36       digitalWrite(LB,LOW);
37       digitalWrite(LF,LOW);
38       digitalWrite(RB,LOW);
39     }
40     if(incoming == 'B'){ // back
41       digitalWrite(RF,LOW);
42       digitalWrite(LB,HIGH);
43       digitalWrite(LF,LOW);
44       digitalWrite(RB,HIGH);
45       analogWrite(l_pwm,speed);
46       analogWrite(r_pwm,speed);
47     }
48     if(incoming == 'F'){
49       digitalWrite(RF, HIGH);
50       digitalWrite(LB, LOW);
51       digitalWrite(LF, HIGH);
52       digitalWrite(RB, LOW);
53       analogWrite(l_pwm, speed);
54       analogWrite(r_pwm, speed);
55     }
56     if (incoming == 'H'){ // back left
57       digitalWrite(RF, HIGH);
58       digitalWrite(LB, HIGH);
59       digitalWrite(LF, LOW);
60       digitalWrite(RB, LOW);
61       analogWrite(l_pwm, speed);
62       analogWrite(r_pwm, speed);
63     }
64     if(incoming == 'L'){ // left
65       digitalWrite(RF, HIGH);
66       digitalWrite(LB, LOW);
67       digitalWrite(LF, HIGH);

```

```

68     digitalWrite(RB, LOW);
69     analogWrite(l_pwm, 150);
70     analogWrite(r_pwm, 255);
71 }
72 if(incoming == 'G'){ //forward left
73     digitalWrite(RF,HIGH);
74     digitalWrite(LB,LOW);
75     digitalWrite(LF,HIGH);
76     digitalWrite(RB,LOW);
77     analogWrite(l_pwm,150);
78     analogWrite(r_pwm,255);
79 }
80 if(incoming == 'J'){ //backright
81     digitalWrite(RF,LOW);
82     digitalWrite(LB,LOW);
83     digitalWrite(LF,HIGH);
84     digitalWrite(RB,HIGH);
85     analogWrite(l_pwm,speed);
86     analogWrite(r_pwm,speed);
87 }
88 if(incoming == 'R'){ // right
89     digitalWrite(RF,HIGH);
90     digitalWrite(LB,LOW);
91     digitalWrite(LF,HIGH);
92     digitalWrite(RB,LOW);
93     analogWrite(l_pwm,255);
94     analogWrite(r_pwm,150);
95 }
96 if (incoming == 'I'){ // forward right
97     digitalWrite(RF,HIGH);
98     digitalWrite(LB,LOW);
99     digitalWrite(LF,HIGH);
100    digitalWrite(RB,LOW);
101    analogWrite(l_pwm,255);
102    analogWrite(r_pwm,190);
103 }
104 if(incoming == '1'){
105     speed=150;
106 }
107 if(incoming == '2'){
108     speed=160;
109 }
110 if(incoming == '3'){
111     speed=170;
112 }
113 if(incoming == '4'){
114     speed=180;
115 }
116 if(incoming == '5'){
117     speed=190;
118 }
119 if(incoming == '6'){
120     speed=200;
121 }
122 if(incoming == '7'){
123     speed=210;
124 }
125 if(incoming == '8'){

```

```

126     speed=220;
127 }
128 if(incoming == '9'){
129     speed=235;
130 }
131 if(incoming == 'q'){
132     speed=255;
133 }
134 }
135 fflush(stdin);
136 }

```

Listing 1: ESP32 Bluetooth Control Firmware

6 Kinematics and Steering

The car employs **Differential Steering**. For a standard forward move ('F'), both motor sets receive equal PWM. For steering maneuvers (e.g., 'G' for forward-left or 'I' for forward-right), the code creates a "speed differential" where one side of the vehicle rotates faster than the other, creating a turning radius without a dedicated steering rack.

7 Conclusion

Phase One successfully establishes a functional Bluetooth-controlled mobile platform. The integration of the ESP32 allows for responsive control and variable speed handling. Future iterations will focus on integrating ultrasonic sensors for autonomous obstacle avoidance and camera modules for First-Person View (FPV) navigation.